TECHNICAL DOCUMENTATION

reverse-workflow-service

Orchestration Service for the LLM-Based Reverse Workflow Recommendation System on the Shuffle SOAR Platform

Field	Value
Component	reverse-workflow-service (Node.js / Express orchestration service)
Version	1.0.0
Runtime	Node.js 20 (node:20-slim), CommonJS
Service port	5005
Role in system	Pipeline orchestrator, knowledge-graph writer, deterministic reverse assembler, three-level validator, Shuffle importer
Upstream dependencies	Shuffle SOAR API, llm-service (Python/FastAPI), Neo4j, PostgreSQL
Document type	Prototype technical documentation — design, interfaces, algorithms, limitations
Document status	Derived directly from the source tree; behaviour statements reflect code as implemented

Table of Contents

1. Introduction

1.1 Purpose of This Document

This document describes the design and implementation of `reverse-workflow-service`, the Node.js orchestration component of a prototype system that automatically derives *reverse workflows* (rollback playbooks) from executed incident-response playbooks on the Shuffle SOAR platform.

It is written to be sufficient for three audiences: a reviewer who needs to understand what the prototype does and how far its claims extend; a developer who needs to run, extend, or debug the service; and an examiner who needs to trace a specific behaviour back to a specific line of reasoning in the code.

Every behavioural statement in this document is derived from the source tree as shipped. Where the code does something surprising, incomplete, or inconsistent with what the module names suggest, the document says so rather than describing the intended design. Those cases are collected in Chapter 11.

1.2 Problem Statement

A SOAR playbook that responds to an incident performs state-changing actions on external systems: it adds firewall address objects, isolates endpoints, disables user accounts, creates detection rules. When the triggering alert turns out to be a false positive, or when the incident is closed, those changes must be undone. In practice this is done manually, which is slow, error-prone, and dependent on the responder remembering exactly what was changed.

A reverse workflow is a second playbook whose actions undo the effects of the first, executed in the opposite order. Producing one automatically is harder than it looks, because:

- **Not every action is reversible.** `post_reset_password` and `post_revoke_user_sessions` change state irrecoverably. Sending an email cannot be un-sent.
- **Reversibility is not inferable from the action name alone.** `patch_enable_or_disable_user` is a single action whose direction lives in a request parameter, not in its name.
- **Reversal frequently requires pre-execution state that was never captured.** Undoing `put_update_firewall_policy` requires knowing the policy before the update; the executed workflow did not record it.
- **A wrong reversal is worse than none.** A generated rollback that deletes the wrong firewall rule causes an outage. Any system that guesses must make its guesses visible.

1.3 Design Position

The service is deliberately **deterministic-first**. The reverse workflow is assembled by rule-based logic operating over a curated reverse-action knowledge base and a Neo4j knowledge graph. The Large Language Model is invoked once per generation attempt, but its output is treated as a *candidate pool* rather than as the workflow itself: candidates are consumed only for source actions that rule-based resolution has explicitly classified as `needs_llm`, and every action derived from an LLM candidate is flagged `requires_manual_review: true`.

> **Framing note.** The system is not an LLM that writes rollback playbooks. It is a deterministic assembler with an LLM fallback for a bounded, explicitly labelled subset of actions. Chapter 7 documents the exact decision points where the LLM output can and cannot influence the result.

The second design commitment is **safe degradation**. When the system cannot determine a safe reversal, it does not omit the action silently and does not invent one. It emits a *checkpoint node* — a no-op `repeat_back_to_me` action carrying a human-readable explanation, flagged for manual review, accompanied by an annotated canvas comment stating the reason, the impact of skipping it, and a recommended manual remedy. The generated workflow is therefore always a draft for analyst review, never an auto-executable artefact.

1.4 Scope and Boundaries

In scope for this document	Out of scope
The Node.js orchestration service: HTTP surface, pipeline stages, graph construction, deterministic assembly, validation, Shuffle import	Internal implementation of `llm-service` (Python/FastAPI): prompt construction, RAG retrieval, embedding and reranking pipeline, model serving
The reverse-action knowledge base (`reverseActionMap.json`) schema, resolution order, and coverage	Shuffle SOAR platform internals and its workflow execution engine
The Neo4j graph schema written and read by this service	Neo4j and PostgreSQL cluster administration, backup, and tuning
Validation rules and their error taxonomy	End-to-end evaluation results and test scenario outcomes (covered in the thesis document)

1.5 Terminology

Term	Definition as used in this document
Source workflow	The executed incident-response playbook submitted to the service for reversal.
Reverse workflow	The generated rollback playbook. Always a draft requiring analyst review.
Source action / source node	One action in the source workflow, as parsed into an internal node object.
Reverse resolution	The rule-based decision assigning one of four statuses to a source action (Chapter 6).
Knowledge graph	The Neo4j property graph holding workflows, actions, apps, action templates, and reverse mappings. Distinct from the reverse-action knowledge base, which is a JSON file.
Reverse-action knowledge base	`config/reverseActionMap.json` — the curated, versioned mapping of app + action to reverse action or non-reversible status.
Checkpoint node	A no-op action emitted in place of an action that has no safe automatic reversal, flagged for manual review.
Candidate pool	The list of actions returned by the LLM service for one generation attempt, before deterministic filtering.

Term	Definition as used in this document
Containment action	A source action classified as state-changing on a security control (the default classification).

2. System Overview

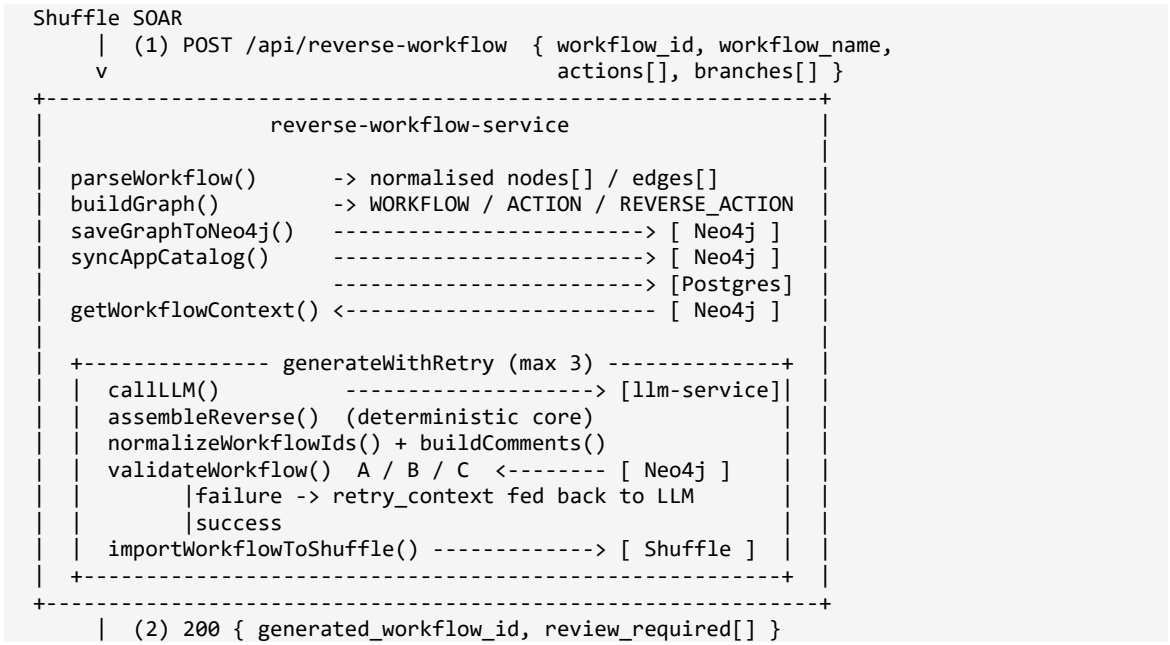
2.1 Deployment Context

The service runs as one container in a multi-container Docker Compose application. It holds no persistent state of its own; all durable state lives in Neo4j, PostgreSQL, and Shuffle.

Peer component	Direction	Protocol	Purpose
Shuffle SOAR	Inbound	HTTP POST	Submits the executed workflow (actions + branches) to `/api/reverse-workflow`.
Shuffle SOAR	Outbound	REST + Bearer token	App catalog retrieval (`GET /api/v1/apps`) and reverse workflow import (`POST /api/v1/workflows`).
Shuffle Cloud	Outbound	REST + Bearer token	Optional additional app catalog source (`POST /api/v1/apps/search`, paginated).
llm-service	Outbound	REST + JWT Bearer	`POST /api/v1/auth/login` for a token, then `POST /api/v1/generate/reverse` for candidate actions.
Neo4j	Bidirectional	Bolt	Writes the workflow knowledge graph and app catalog; reads them for validation and context checks.
PostgreSQL	Outbound	TCP (pg pool)	Secondary relational mirror of the app catalog. Written on sync; never read by this service.

Observation. PostgreSQL is written but never read anywhere in this service. It is a mirror maintained for external inspection or for other components; the pipeline does not depend on it. A failure to write to PostgreSQL will abort the catalog sync (the error is rethrown by `saveAppsToPostgres`) but the sync itself is wrapped in `safeSyncApps`, so the pipeline continues.

2.2 High-Level Data Flow



```
      v
Shuffle SOAR / analyst
```

Figure 2.1 — End-to-end request flow through the orchestration service.

2.3 Repository Layout

The service is organised by pipeline stage rather than by technical layer; each directory corresponds to one step in Figure 2.1.

Path	LOC	Responsibility
`server.js`	143	Express application, three API endpoints, in-memory status store, pipeline sequencing, startup.
`parsers/workflowParser.js`	39	Normalises the raw Shuffle payload into internal `nodes[]` and `edges[]` with defaulted fields.
`graph/graphBuilder.js`	116	Builds the property-graph representation: WORKFLOW, ACTION, REVERSE_ACTION nodes and their relationships. Resolves reverse status per action.
`neo4j/neo4jDriver.js`	22	Singleton Neo4j driver with connectivity verification at load time.
`neo4j/saveGraph.js`	91	Persists the graph via label-specific MERGE statements.
`neo4j/queryWorkflowContext.js`	48	Retrieves the stored workflow context (workflow, actions, apps, reverse map).
`config/reverseMap.js`	81	Loader and resolver for the reverse-action knowledge base; implements the four-status resolution order.
`config/reverseActionMap.json`	718	The curated knowledge base itself: 18 applications, 98 reversible pairs, heuristic prefix rules.
`llm/llmService.js`	629	LLM authentication and invocation, the deterministic assembly algorithm, canvas comment generation, and the retry loop.
`validators/validateWorkflow.js`	575	Three-level validation (structural, semantic, reverse-mapping), correction-instruction synthesis, graph helper queries.
`builders/buildShuffleWorkflow.js`	232	Shuffle import client. Also contains an unused workflow compiler — see Section 11.1.
`shuffleApps/syncAppCatalog.js`	87	TTL-gated catalog synchronisation orchestrator.
`shuffleApps/fetchApps.js`	118	Paginated retrieval of apps from Shuffle Cloud and local Shuffle, with de-duplication.
`shuffleApps/saveAppsToNeo4j.js`	133	Writes APP, ACTION_TEMPLATE, PARAMETER_TEMPLATE nodes and the sync metadata singleton.
`shuffleApps/saveAppsToPostgres.js`	157	Writes the relational mirror inside a single transaction.
`shuffleApps/postgresClient.js`	18	PostgreSQL connection pool.
`utils/paperLog.js`	127	Structured console reporting of parsing, prompt/output, validation, and import stages.
`Dockerfile`	15	Node 20 slim image, production dependency install, port 5005.

Table 2.1 — Module inventory. Line counts exclude dependencies.

2.4 Runtime Dependencies

Package	Version	License	Used for
express	5.2.1	MIT	HTTP server and routing.
axios	1.17.0	MIT	Outbound HTTP to llm-service, Shuffle, and Shuffle Cloud.
neo4j-driver	6.0.1	Apache-2.0	Bolt connectivity and Cypher execution.
pg	8.21.0	MIT	PostgreSQL connection pooling.
cors	2.8.6	MIT	Cross-origin support (currently unrestricted).
dotenv	17.4.2	BSD-2-Clause	Environment variable loading from the compose-root `.env`.
uuid	9.0.1	MIT	UUID v4 generation inside `builders/`.
nodemon	3.1.14	MIT	Development auto-reload. Declared as a runtime dependency — see Section 11.9.

The lockfile resolves 135 packages in total. `llm/llmService.js` uses Node's built-in `crypto.randomUUID()` rather than the `uuid` package; the two coexist because they were introduced in different modules.

3. Configuration and Deployment

3.1 Environment Variables

Configuration is entirely environment-driven. Note the resolution path: every module that calls `dotenv.config()` resolves the file to the **parent directory of the service root**, not to the service root itself. In the Compose layout this is the project root shared by all services.

```
server.js                __dirname/../.env
neo4j/neo4jDriver.js     __dirname/../../.env
llm/llmService.js        __dirname/../../.env
builders/buildShuffleWorkflow.js  __dirname/../../.env
shuffleApps/fetchApps.js          __dirname/../../.env
```

All five paths resolve to the same file one level above the service directory.

Variable	Default	Required	Consumer / effect
`NEO4J_URI`	`bolt://localhost:7687`	Effectively yes	Bolt endpoint. Connectivity is verified at module load; a failure logs a warning and does not stop startup.
`NEO4J_USERNAME`	`neo4j`	Yes	Basic auth user.
`NEO4J_PASSWORD`	`""` (empty)	Yes	Basic auth password. Empty default will fail against any secured instance.
`LLM_API_URL`	`http://localhost:8000`	Yes	Base URL of the Python llm-service.
`LLM_API_PREFIX`	`/api/v1`	No	Path prefix prepended to `/auth/login` and `/generate/reverse`.
`LLM_AUTH_USER`	`admin`	Yes	Credential for the llm-service login endpoint.
`LLM_AUTH_PASS`	`admin`	Yes	Credential for the llm-service login endpoint. The default is a placeholder and must be overridden.
`LLM_CALL_TIMEOUT_MS`	`960000` (16 min)	No	Axios timeout for generation. Sized for local CPU inference.
`SHUFFLE_API_URL`	(none)	Yes	Local Shuffle base URL. Used for app catalog retrieval and workflow import.
`SHUFFLE_API_KEY`	`""`	Yes	Bearer token. `importWorkflowToShuffle` throws immediately if empty.
`SHUFFLE_CLOUD_URL`	(none)	No	If unset, cloud catalog retrieval is skipped with a warning.
`SHUFFLE_CLOUD_API_KEY`	`""`	No	Bearer token for Shuffle Cloud.
`APP_SYNC_TTL_HOURS`	24	No	Catalog freshness window. A sync older than this triggers a refresh.
`POSTGRES_HOST`	(none)	For mirror	Passed straight to the pg Pool.
`POSTGRES_PORT`	(none)	For mirror	Passed as a string; pg coerces it.

Variable	Default	Required	Consumer / effect
POSTGRES_DB	(none)	For mirror	Database name.
POSTGRES_USER	(none)	For mirror	Role name.
POSTGRES_PASSWORD	(none)	For mirror	Role password.

Table 3.1 — Complete environment variable reference.

Not configurable. The listening port is hard-coded to `5005` in `server.js` (`app.listen(5005, ...)`). There is no `PORT` variable. Changing the port requires a code edit or a container port mapping.

3.2 Container Image

```
FROM node:20-slim
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 5005
CMD ["npm", "start"]
```

Figure 3.1 — `Dockerfile`. Dependency layer is copied before source to preserve build cache.

`.dockerignore` excludes `node_modules`, `npm-debug.log`, `.env`, and `.git`. Because `.env` is excluded from the image and the code resolves it to the parent of `/app`, the file must be supplied at runtime — by bind mount or by Compose `env_file` / `environment`.

3.3 Startup Sequence

1. `dotenv` loads the environment before any other module is required.
2. Requiring `neo4j/neo4jDriver.js` constructs the driver and fires `verifyConnectivity()` asynchronously. Failure logs `[neo4j] Warning: ...` and startup continues — the service will start against an unreachable database.
3. Express middleware is registered: unrestricted CORS, JSON body parser with a 10 MB limit, URL-encoded parser with the same limit.
4. `startServer()` awaits `safeSyncApps()`, which performs a TTL-gated catalog synchronisation. Any error is caught and logged as `[APP] sync skipped`.
5. `app.listen(5005)` binds the port.

Consequence. Because catalog sync is awaited before `listen()`, a cold start against an empty Neo4j blocks until the full Shuffle catalog has been fetched and written. With a large catalog this can be a multi-minute startup. Health checks configured with a short start period will fail during this window.

3.4 Connection Lifecycle

- **Neo4j** — one module-level driver, pool size 10, connection timeout 10 s. Each operation opens and closes its own session in a `finally` block. There is no explicit driver shutdown on process termination.
- **PostgreSQL** — one module-level `pg.Pool` using library defaults. Clients are acquired per sync and released in a `finally` block.

- **llm-service token** — cached in the module-level variable `_token` after the first successful login. It is never refreshed or invalidated; an expired token will surface as a call failure that is retried by the generation loop but not by re-authentication. See Section 11.5.

4. API Reference

All endpoints are unauthenticated and CORS is unrestricted (`app.use(cors())` with no options). The service is intended to run on an internal network segment; see Section 11.8.

4.1 GET /

Liveness probe. Returns the plain-text string `Reverse Workflow Service Running` with status 200. Performs no dependency checks — it will return 200 while Neo4j, Shuffle, and llm-service are all unreachable.

4.2 POST /api/reverse-workflow

The primary endpoint. Accepts an executed workflow, runs the full pipeline synchronously, and returns the outcome. Request body:

```
{
  "workflow_id":   "string",   // identifier of the source workflow
  "workflow_name": "string",   // display name; used for the reverse workflow name
  "actions": [                  // executed actions
    {
      "id":          "string",
      "label":       "string",
      "name":        "string",   // fallback for action_name
      "action_name": "string",
      "app_name":    "string",
      "app_id":      "string",
      "app_version": "string",
      "category":    "string",
      "environment": "string",
      "isStartNode": false,
      "position":    { "x": 0, "y": 0 },
      "parameters":  [ { "name": "...", "value": "..." } ]
    }
  ],
  "branches": [                 // control-flow edges
    {
      "source_id":      "string",
      "destination_id": "string",
      "label":          "string",
      "conditions":     "string | array"
    }
  ]
}
```

Figure 4.1 — Request schema. Absent fields are defaulted by `parseWorkflow`; see Section 5.1.

4.2.1 Success — reverse workflow generated

```
HTTP 200
{
  "success": true,
  "workflow_id": "<source id>",
  "generated_workflow_id": "<id returned by Shuffle>",
  "generated_workflow_name": "<name returned by Shuffle>",
  "attempts": 1,
  "review_required": [
    { "source_action_name": "...", "app_name": "...",
      "status": "requires_manual_review", "reason": "..." }
  ],
  "message": "Reverse workflow generated successfully (draft – perlu peninjauan analis)"
}
```

4.2.2 Success — nothing to reverse

When every source action classifies as read-only or utility, the assembler produces zero actions. This is a **correct success outcome**, not a failure: there is nothing to roll back, so no workflow is imported and `generated_workflow_id` is `null`. The response carries `note: "no_auto_reverse"`.

```
HTTP 200
{
  "success": true,
  "workflow_id": "<source id>",
  "generated_workflow_id": null,
  "generated_workflow_name": null,
  "attempts": 1,
  "review_required": [],
  "note": "no_auto_reverse",
  "message": "Tidak ada aksi yang memerlukan pembalikan (seluruh aksi read-only/utilitas)."
}
```

Implementation detail. `server.js` guards this branch with `if (!llmResult.importResult || llmResult.note === "no_auto_reverse")` before dereferencing `importResult.id`. The guard exists because the earlier version crashed on this path.

4.2.3 Failure

```
HTTP 500
{ "success": false, "error": "<message>" }
```

Trigger	`error` message shape
Workflow not persisted or not retrievable from Neo4j	`Workflow context not found in Neo4j`
All three generation attempts exhausted	`Failed after 3 attempts → [ ...serialised error objects... ]`
Malformed payload (e.g. `actions` not an array)	TypeError message propagated from `parseWorkflow`

Status code caveat. Every failure returns 500, including client-side faults such as a malformed body. There is no 400 path on this endpoint.

4.3 GET /api/reverse-workflow/status/:id

Returns the last recorded status for a source `workflow_id`. Status transitions are `processing` → `success` | `failed`.

```
HTTP 200
{ "success": true, "workflow_id": "...", "status": "success",
  "updated_at": "2026-07-29T04:15:22.010Z",
  "generated_workflow_id": "...", "review_required": [ ... ] }

HTTP 404
{ "success": false, "error": "workflow_id tidak ditemukan" }
```

Volatility. The status store is a plain in-memory `Map`. It is lost on restart, is not shared between replicas, and grows without bound — entries are never evicted. Because the main endpoint is synchronous and only returns after the pipeline completes, this endpoint is of limited practical use in the current design; it is primarily useful for post-hoc inspection within a single process lifetime.

4.4 POST /api/validate-workflow

Exposes the validator independently of generation, for testing a candidate workflow JSON against the three validation levels.

```
Request:  { "workflow": { ...Shuffle workflow JSON... },
            "workflow_id": "<optional source id>" }

HTTP 200  { "success": true, "valid": true, "errors": [],
            "review_required": [ ... ] }
HTTP 200  { "success": true, "valid": false, "errors": [ ... ],
            "correction_instructions": "..." }
HTTP 400  { "success": false, "error": "field 'workflow' wajib diisi" }
HTTP 500  { "success": false, "error": "<message>" }
```

Omitting `workflow_id` skips Level C entirely (`validateReverseMapping` returns immediately when the id is null) and causes `review_required` to be empty.

5. Processing Pipeline

This chapter walks the pipeline stage by stage in execution order, as sequenced by the `POST /api/reverse-workflow` handler.

5.1 Stage 1 — Parsing

`parseWorkflow(actions, branches)` performs field normalisation only; it applies no semantics. Its purpose is to guarantee that downstream stages can assume a fixed shape.

Output field	Source	Fallback
`id`	`action.id`	`""`
`label`	`action.label`	`action.name`
`action_name`	`action.action_name`	`action.name`, then `""`
`app_name`, `category`, `app_id`, `app_version`, `environment`	same-named fields	`""`
`position`	`action.position`	`{ x: 0, y: 0 }`
`isStartNode`	`!!action.isStartNode`	`false`
`parameters`	`action.parameters` if an array	`[]`

Edges are reduced to `{ source, target, label, conditions }`, mapping `source_id` and `destination_id` respectively. The parser does not validate that edge endpoints reference existing actions.

Immediately after parsing, `paperLog.logParsing()` prints a Markdown-formatted table of the parsed workflow to stdout, labelled `[4.2.3] HASIL PARSING WORKFLOW`. The bracketed prefixes in `paperLog` correspond to section numbers in the thesis document, so console output can be transcribed directly into the results chapter.

5.2 Stage 2 — Graph Construction

`buildGraph(parsed, workflowId, workflowName)` converts the flat parse result into a property-graph description. Three things happen here that matter downstream.

5.2.1 Start-node inference

The start node is the first parsed node whose id never appears as an edge target. This is a pure in-degree-zero heuristic: it ignores the `isStartNode` flag carried in the payload, and if the graph has several roots it silently selects whichever appears first in the action array.

5.2.2 Reverse resolution at graph-build time

For every source action, `resolveReverse(action_name, app_name)` is called and its result is materialised as a dedicated `REVERSE_ACTION` node with id `REV_<action_id>`, linked by `HAS_REVERSE`. **The reverse**

decision is therefore made and persisted before the LLM is ever contacted. The graph records the decision, its status, and its reason, which is what makes the resolution auditable after the fact.

5.2.3 Role inference

`inferRole(node)` assigns one of `TRIGGER`, `RESPONSE`, `ANALYSIS`, or `ENRICHMENT` by substring matching on the label. It is stored as the `role` property on every ACTION node.

> **Dead property.** `role` is written to Neo4j but never read by any Cypher query or JavaScript module in this service. The classification that actually drives assembly is `classifyAction()` in `llm/llmService.js`, which is a different, more detailed taxonomy (Section 7.2). If `role` is described as functional in the thesis, that claim should be corrected: it is descriptive metadata, not control flow.

5.2.4 Emitted graph elements

Element	Cardinality	Notes
`WORKFLOW` node	1	Carries `workflow_id`, `workflow_name`, `description`, `start_node`.
`ACTION` node	1 per source action	`position` and `parameters` are JSON-stringified before storage.
`REVERSE_ACTION` node	1 per source action	Always created, including for `no_reverse_needed` actions.
`CONTAINS` relationship	1 per action	WORKFLOW → ACTION.
`USES_APP` relationship	1 per action with a non-empty `app_id`	ACTION → APP.
`HAS_REVERSE` relationship	1 per action	ACTION → REVERSE_ACTION, carrying `status`.
`NEXT` relationship	1 per branch	ACTION → ACTION, carrying the serialised condition.

5.3 Stage 3 — Persistence

`saveGraphToNeo4j(graph)` writes nodes first, then relationships. Nodes use label-specific `MERGE` on the natural key (`workflow_id`, `action_id`, `app_id`, `rev_id`) followed by `SET n += $props`, which makes the operation idempotent: re-submitting the same workflow updates rather than duplicates.

Property values are passed through `sanitizeProps`, which JSON-stringifies any object and preserves `null`. Neo4j cannot store nested maps as property values, so this coercion is required.

Relationships are created by a single parameterised Cypher statement that matches endpoints across all four labels by their respective key properties, then `MERGE`s the typed relationship. The relationship type is interpolated into the query string (`MERGE (a)-[r:${rel.type}]->(b)`) because Cypher does not permit a parameterised relationship type; the value originates from a fixed set of literals inside `buildGraph` and is never user-controlled.

> **Error handling defect.** The `try` block ends in `catch (error) { console.error(...) }` with no rethrow. A failed graph write is logged and the function resolves normally. The pipeline then proceeds

to `getWorkflowContext`, which finds nothing and throws `Workflow context not found in Neo4j`. The reported error therefore points at the retrieval step rather than the write that actually failed. See Section 11.3.

5.4 Stage 4 — App Catalog Synchronisation

`safeSyncApps()` wraps `syncAppCatalog()` so that a catalog failure degrades the request rather than failing it. The sync itself is TTL-gated:

6. `checkSyncNeeded()` reads the `APP_SYNC_META {id: "singleton"}` node and counts `APP` nodes.
7. A sync is forced if no metadata exists, if the app count is zero, if the last sync timestamp is missing, or if it is older than `APP_SYNC_TTL_HOURS`.
8. Any error during the check forces a sync rather than skipping one — fail-safe rather than fail-open.
9. `fetchAllApps()` retrieves from Shuffle Cloud first, then local Shuffle, paginating at 200 items per page up to 50 pages (a 10 000-app ceiling). Local apps whose `id` already appeared in the cloud result are discarded, so **cloud definitions win on collision**.
10. Results are written to Neo4j and then PostgreSQL. An empty catalog aborts the sync with `reason: "empty_catalog"` rather than wiping existing data.

Ordering issue. In the request handler, `safeSyncApps()` runs **after** `saveGraphToNeo4j()`. The `USES_APP` relationships written in stage 3 match on `(b:APP {app_id: ...})`; if the catalog has not yet been populated, those `MATCH` clauses find nothing and the `MERGE` silently creates no relationship, with no error. In practice the boot-time sync usually populates the catalog first, which masks the problem. See Section 11.4.

5.5 Stage 5 — Context Verification

`getWorkflowContext(workflow_id)` runs one Cypher query that collects the workflow node, its actions, the apps those actions use, and the reverse-mapping projections. If no record is returned, the handler throws.

Unused return value. The returned `context` object is checked for existence and then discarded — it is never passed to the LLM call or to the assembler. The retrieval that actually feeds the prompt happens inside the Python `llm-service`, which queries Neo4j independently using the `workflow_id`. In this service the call functions purely as a persistence barrier: it confirms the graph write landed before the LLM is asked to read it.

5.6 Stage 6 — LLM Invocation

`callLLM({ workflow_id, workflow_name, retryContext })` posts a deliberately minimal body: only identifiers and, on a retry, the previous validation errors. It does **not** send the workflow content — the llm-service retrieves that from Neo4j itself.

```
POST {LLM_API_URL}{LLM_API_PREFIX}/generate/reverse
Authorization: Bearer <token from /auth/login>

{ "workflow_id":   "...",
  "workflow_name": "...",
  "retry_context": null | {
```

```
      "attempt": 2,
      "valid": false,
      "errors": [ { "code": "...", "location": "...", "message": "..." } ],
      "correction_instructions": "..." } }
```

Authentication is a single `POST /auth/login` with `LLM_AUTH_USER` / `LLM_AUTH_PASS`, returning `access_token`, which is cached in module scope for the process lifetime.

The response is unwrapped through a fallback chain — `data.raw_output || data.workflow || data.result || data.output || data` — and the accompanying `data.prompt`, when present, is captured for logging. `parseWorkflowJSON` accepts either an object or a string, stripping Markdown code fences before `JSON.parse`.

> **Fallback risk.** The final `|| data` term means an unrecognised response envelope is passed through as though it were the workflow. It would then fail parsing or assembly rather than being rejected with a clear message. A strict check on the expected key would fail faster and more legibly.

5.7 Stages 7–9 — Assembly, Normalisation, Validation, Import

These stages are the substance of the contribution and are documented separately: assembly in Chapter 7, validation in Chapter 8. In sequence within one attempt:

11. `assembleReverse(workflow_name, sourceNodes, llmActions)` produces the ordered action list and the `review_required` list.
12. If the action list is empty, the attempt returns immediately with `note: "no_auto_reverse"`. **No validation and no import occur on this path.**
13. `normalizeWorkflowIds(workflow)` reassigns UUIDs, forces a single start node, rebuilds branches as a linear chain, and appends `" (reverse)"` to the name if not already present.
14. `getAppImages(appIds)` fetches `large_image` values from the APP nodes and injects them into each action, so that the semantic validator's image comparison can succeed. Failure here is logged and tolerated.
15. `buildComments()` computes the rollback coverage summary and generates canvas comments; the summary headline is prepended to the workflow description.
16. `validateWorkflow(workflow, workflow_id)` runs levels A, B, and C in order, short-circuiting at the first level that produces errors.
17. On success, `importWorkflowToShuffle(workflow)` POSTs to `{SHUFFLE_API_URL}/api/v1/workflows` with the bearer token and returns Shuffle's response body.

5.8 The Retry Loop

`generateWithRetry` wraps stages 6–9 in a bounded loop of `maxRetries` (default 3) attempts. The loop is a closed feedback cycle: whatever caused the previous attempt to fail is serialised into `retry_context` and sent back to the llm-service on the next call.

Failure point	Recorded code	Loop behaviour
LLM call throws (network, timeout, HTTP error, service error field)	`LLM_CALL_ERROR`	`continue` — next attempt carries the error text as retry context.
Response is not parseable JSON, or assembly throws	`INVALID_JSON`	`continue` — next attempt carries the parse error.
Validation returns `valid: false`	Level A/B/C codes	`continue` — next attempt carries the full error array plus synthesised correction instructions.
Shuffle rejects the import	`SHUFFLE_IMPORT_ERROR`	`continue` — built by `buildImportError()` from the rejection message.
Assembly yields zero actions	—	Immediate successful return with `no_auto_reverse`; the loop does not continue.
Validation passes and import succeeds	—	Return `{ workflow, importResult, attempts, review_required }`.

Table 5.1 — Retry loop outcomes.

Exhausting all attempts throws an error whose message embeds the serialised errors from the final attempt, which `server.js` surfaces as the HTTP 500 body.

> **Feedback asymmetry.** `LLM_CALL_ERROR` is fed back to the llm-service as if it were a correctable output defect, but a connection timeout is not something the model can fix by regenerating. The loop treats infrastructure faults and content faults identically. Distinguishing them would let transport failures be retried without consuming a generation attempt.

6. The Reverse-Action Knowledge Base

6.1 Rationale

Reversibility is a property of a specific action in a specific application, and it is not reliably derivable from the action's name. `post_add_firewall_address` has a clean inverse in FortiGate; `post_block_outbound_ip` has none in Palo Alto Networks because the connector exposes no unblock operation; `patch_enable_or_disable_user` is its own inverse with a flipped parameter. Encoding this as curated data rather than as inference is what allows the pipeline to be deterministic and auditable.

The knowledge base is `config/reverseActionMap.json`, currently at schema `version: 2`, covering **18 applications** and **98 explicit reversible pairs**.

6.2 Schema

```
{
  "version": 2,
  "_doc": { ...human-readable notes on matching and resolution order... },
  "heuristics": {
    "no_reverse_prefixes":     [ "get_", "list_", "search_", ... ],
    "manual_review_prefixes":  [ "delete_", "update_", "patch_", ... ],
    "reversible_prefix_pairs": [ ["post_add_", "delete_"], ... ],
    "needs_llm_actions":       [ "custom_action" ]
  },
  "apps": {
    "<App_Name>": {
      "reversible":             [ { "action": "...", "reverse_action": "..." } ],
      "requires_manual_review": [ "..." ],
      "no_reverse_needed":      [ "..." ],
      "needs_llm":              [ "..." ],
      "default_status":         "no_reverse_needed",
      "_note":                  "rationale for the curation decision"
    }
  }
}
```

Figure 6.1 — Knowledge base schema.

6.3 Resolution Statuses

Status	Meaning	Effect in assembly
`auto_mapped`	A curated inverse action exists for this app + action pair.	Emit the inverse action, reusing the source parameters. Not flagged for review.
`requires_manual_review`	The action is destructive, stateful, or has no exposed inverse.	Emit a checkpoint node and add an entry to `review_required`.
`no_reverse_needed`	Read-only or utility; it changed no external state.	Skip entirely — the action does not appear in the reverse workflow.
`needs_llm`	Not classifiable by rule; the inverse must be inferred from configuration.	Consume an LLM candidate if one matches the app; otherwise emit a checkpoint.

6.4 Resolution Order

`resolveReverse(actionName, appName)` evaluates in strict priority order. Matching normalises both names to lowercase with whitespace and hyphens replaced by underscores, so `Post Add Firewall Address` and `post-add-firewall-address` resolve identically.

18. **Global LLM override.** If the normalised action name is in `heuristics.needs_llm_actions` (currently only `custom_action`), return `needs_llm` immediately. This precedes even app-specific configuration, because a generic HTTP action's inverse depends on its method, URL, and body rather than on its name.
19. **App lookup.** Resolve the app entry by exact key, falling back to a normalised comparison across all keys. This makes `Microsoft Graph` in Neo4j match the `Microsoft_Graph` key in the file.
20. **`reversible`** — exact action match returns `auto_mapped` with the normalised inverse name.
21. **`requires_manual_review`** — exact match, reason `app_listed`.
22. **`no_reverse_needed`** — exact match, reason `app_listed`.
23. **`needs_llm`** — exact match, reason `app_listed`.
24. **`default_status`** — an app-wide fallback, reason `app_default`. Used for whole-app classifications such as `Shuffle Tools` and `email`.
25. **`no_reverse_prefixes`** — prefix match returns `no_reverse_needed`, reason `heuristic_prefix`.
26. **`manual_review_prefixes`** — prefix match returns `requires_manual_review`, reason `heuristic_prefix`.
27. **Terminal default** — `needs_llm`, reason `unclassified`.

> **Terminal default.** An unknown action from an unknown app becomes `needs_llm`, not `requires_manual_review`. If the LLM supplies no matching candidate, `assembleReverse` converts it to a checkpoint anyway (Section 7.6), so the ultimate safety behaviour is preserved — but only through that second guard, not through the resolver itself.

6.5 Coverage

Application	Reversible pairs	Manual review	No reverse	Needs LLM	App default
FortiGate_Firewall	11	1	2	0	—
Palo_Alto_Networks	0	4	0	0	—
Cisco_ASA	1	0	0	0	—
AWS_Network_Firewall	6	4	0	0	—
FortiEDR	5	5	0	0	—
Sophos_Central	0	3	0	0	—
Microsoft_Entra_ID	0	2	0	1	—
Wazuh	13	6	0	0	—
Datadog	26	0	0	0	—
Elasticsearch	4	2	0	0	—

Application	Reversible pairs	Manual review	No reverse	Needs LLM	App default
FortiSIEM	0	5	0	0	—
Microsoft_Graph	0	3	4	1	—
Slack	32	61	9	1	—
Shuffle Tools	0	0	0	0	`no_reverse_needed`
Shuffle AI	0	0	0	0	`no_reverse_needed`
ShuffleHealthcheck	0	0	0	0	`no_reverse_needed`
http	0	0	0	0	`no_reverse_needed`
email	0	0	0	0	`no_reverse_needed`
Total	**98**	**96**	**15**	**3**	5 apps

Table 6.1 — Knowledge base coverage by application.

Two curation patterns are worth noting because they demonstrate that the base encodes judgement rather than just string pairs:

- **Palo Alto Networks and Sophos Central have zero reversible pairs by design.** The `_note` fields record why: the connectors expose no inverse operation, so every state-changing action is routed to manual review rather than to a guessed inverse.
- **Microsoft Graph lists four read actions explicitly under `no_reverse_needed`.** The connector prefixes read operations with `post_` (`post_microsoft_graph_api_get_user`), which defeats the `get_` prefix heuristic. Without the explicit listing these would fall through to `needs_llm` and pull in the LLM unnecessarily.

6.6 Unused Heuristic

`heuristics.reversible_prefix_pairs` defines twelve generic inverse prefix pairs (`post_add_` ↔ `delete_`, `block_` ↔ `unblock_`, `isolate_` ↔ `unisolate_`, `grant` ↔ `revoke`, and so on). **No code in this service reads that key.** `resolveReverse` consults only `needs_llm_actions`, `no_reverse_prefixes`, and `manual_review_prefixes`.

> **Status.** This is either a documented extension point for future automatic pair inference, or data consumed by the Python llm-service as prompt context. Within this service it is inert. It should not be described as an active resolution mechanism.

7. The Deterministic Assembly Algorithm

`assembleReverse()` in `llm/llmService.js` is the core of the contribution. It takes the source nodes and the LLM candidate pool and returns the ordered action list plus the manual-review register. Everything it does is deterministic given the same inputs, apart from UUID generation.

7.1 Reverse Chronological Ordering

The algorithm iterates `[...sources].reverse()` — last-executed action first. This is a correctness requirement, not a presentational choice. If a playbook creates an address object and then references it from a policy, the rollback must delete the policy before the address object; deleting in forward order would fail on the dependency.

> **Ordering caveat.** Reverse chronological order is the correct default but not universally correct — some dependency chains are not the exact inverse of execution order. The generated workflow is a draft precisely so that an analyst can reorder where necessary.

7.2 Action Classification

`classifyAction(node)` assigns each source action to one of five categories. Evaluation is ordered, first match wins.

#	Category	Test	Assembly treatment
1	`utility`	app_name contains “shuffle tools”, or the action name matches `execute_python`, `repeat_back_to_me`, `parse_*`, `regex`, `set_variable`, `wait`.	Skipped. No external state was changed.
2	`read`	Action name begins with `get_`, `list_`, `describe_`, `search_`, `fetch_`, `read_`, `lookup_`, `find_`; **or** a parameter named `method` has the value `GET`.	Skipped.
3	`audit_log`	App is one of nine SIEM/observability platforms *and* the action name matches a write verb; or the label matches `record|audit|log|siem` with a write verb.	Preserved and appended to the tail, with rollback annotation.
4	`notification`	App is one of fourteen messaging/ticketing platforms; or the action name matches `send_`, `post_message`, `notify`, `create_ticket`, `create_issue`, `create_incident`, `create_alert`.	Preserved and appended to the tail, with rollback annotation.
5	`containment`	Default — everything not matched above.	Subject to full reverse resolution.

Table 7.1 — Source action classification taxonomy.

The `method: GET` test is what makes generic `http` actions classify correctly: an HTTP action named `custom_action` carries its semantics in its parameters, not its name.

7.2.1 Why observability actions are preserved rather than reversed

A rollback should not delete the SIEM record of the original response, and it cannot un-send a notification. Instead, both are re-emitted with their content rewritten to describe the rollback. `withRollbackContent()` locates the first parameter named `body`, `message`, `data`, `text`, `content`, or

`description`; if its value parses as JSON it injects `event: "rollback"`, `rollback: true`, a message, and `source_workflow`, preserving the rest of the payload. If it is not JSON, the value is replaced with the plain message. If no such parameter exists, a body parameter is appended.

The effect is that the reverse workflow ends by writing an audit record of its own execution and notifying the same channels that were notified originally — closing the incident loop rather than erasing it.

7.3 Parameter-Toggle Reversal

Before consulting the knowledge base, `buildToggleReverse(node)` tests whether the action is self-inverse under a boolean flip. This handles the `patch_enable_or_disable_user` pattern, where a single action's direction lives in its payload.

```
TOGGLE_BOOL_KEYS = [ accountenabled, account_enabled, enabled, disabled,
                     isenabled, is_enabled, active, blocked, is_blocked,
                     isblocked, suspended ]

1. If a parameter named 'body' holds JSON, flip any top-level key
   whose lowercased name is in TOGGLE_BOOL_KEYS.
2. Independently, flip any parameter whose own name is in the list.
3. If nothing was flipped, return null and fall through to the
   knowledge base.
4. If something was flipped, emit the SAME action name with the
   flipped payload, labelled 'Reverse: <original label>'.
```

Figure 7.1 — Toggle detection. `flipBoolValue` accepts booleans and the strings “true”/“false”; anything else returns undefined and is left untouched.

> **Precedence.** The toggle test runs **before** `resolveReverse`. An action with a flippable boolean is therefore reversed by toggle even if the knowledge base classified it `requires_manual_review`. This is intentional for the enable/disable case but is a broad rule: any action carrying a parameter named `active` or `enabled` will be toggled and marked auto-reversed without review. Section 11.6 discusses the exposure.

7.4 The Assembly Decision Sequence

For each source action, in reverse execution order:

```
cat = classifyAction(node)

  cat in {utility, read}          -> skip
  cat in {audit_log, notification}-> defer to observability tail

  toggle = buildToggleReverse(node)
  toggle != null                  -> emit toggle, mark reverted

  rev = resolveReverse(action_name, app_name)

  rev.status == no_reverse_needed -> skip
  rev.status == auto_mapped       -> emit rev.reverse_action_name
                                     with the source parameters
  rev.status == needs_llm         -> if isReadOnlySource(node): skip
                                     else consume an LLM candidate
                                     matching app_name, or emit a
                                     checkpoint if none matches
  otherwise                       -> emit a checkpoint

actions = [ ...containment, ...observability_tail ]
```

Figure 7.2 — Per-action decision sequence.

7.5 LLM Candidate Consumption

This is the only point at which model output influences the result. When resolution returns `needs_llm`:

28. `isReadOnlySource(node)` runs as a second read-only guard, catching read actions that the app-level configuration routed to `needs_llm`. If it matches, the action is skipped without consulting the pool.
29. The pool is searched for the first candidate whose `app_name` equals the source action's `app_name`, case-insensitively.
30. A match is removed from the pool with `splice` — each candidate is consumed at most once.
31. The emitted action takes its `name` and `label` from the candidate but its `app_id` and `app_version` from the **source node**, so the model cannot introduce an app identity that is not already present in the workflow. Parameters come from the candidate if it supplied a non-empty array, otherwise from the source node.
32. The action is flagged `requires_manual_review: true` unconditionally.
33. If no candidate matches, a checkpoint is emitted and the action is registered in `review_required` with reason `LLM tak menghasilkan pembalik`.

> **Matching weakness.** Candidates are matched by `app_name` alone, not by source action id. If a workflow contains two `needs_llm` actions from the same application, the first candidate in the pool is consumed for whichever source action is processed first — which, given reverse iteration, is the last-executed one. There is no verification that the candidate was generated for that specific action. See Section 11.2.

The consequence of this design is a hard bound on model influence: the LLM cannot add actions for source actions the resolver did not mark `needs_llm`, cannot change the ordering, cannot suppress a checkpoint, and cannot introduce an unknown application. Its influence is confined to the action name and parameter values of actions that are, by construction, flagged for human verification.

7.6 Checkpoint Nodes

A checkpoint is a `repeat_back_to_me` action from the Shuffle Tools app, which is a no-op that echoes its input. It is emitted whenever a state-changing action has no safe automatic reversal.

```
label: "CHECKPOINT (manual): tinjau '<action_name>' (<app_name>)
        — <reason>"
requires_manual_review: true
parameters: [ { name: "call", value: <the same label> } ]
```

> **Hard dependency.** `buildCheckpoint` returns `null` when the source workflow contains no Shuffle Tools action, because it copies `app_id` and `app_version` from that node. If the source workflow never used Shuffle Tools, **no checkpoint is emitted at all** — the irreversible action disappears from the reverse workflow with no visible marker on the canvas. The entry still appears in the `review_required` API field, so the information is not lost, but the canvas no longer shows it. This is the most consequential silent-degradation path in the pipeline; see Section 11.7.

7.7 Normalisation

`normalizeWorkflowIds(workflow)` prepares the assembled list for Shuffle:

- Appends `" (reverse)"` to the workflow name unless it already ends that way.
- Assigns a fresh UUID v4 to every action, so ids can never collide with the source workflow.
- Defaults `requires_manual_review` to `false`, `execution_delay` to `0`, `large_image` to `""`.
- Assigns canvas coordinates on a vertical column: `x = 300`, `y = 150 + index × 150`.
- Sets `is_start_node` true on the first action only, and `workflow.start` to its id.
- **Discards any branch structure and rebuilds `branches` as a strict linear chain** — action *i* to action *i+1* with an empty condition.

> **Structural limitation.** The reverse workflow is always linear. Conditional branches, parallel paths, and loops present in the source workflow are not represented in the reverse workflow. For rollback playbooks this is defensible — undo steps are usually sequential — but it is a genuine expressiveness limit and should be stated as such rather than left implicit.

7.8 Rollback Coverage Summary and Canvas Annotations

After normalisation, `buildComments()` classifies each **emitted** action — a different taxonomy from `classifyAction`, operating on output rather than input.

Emitted class	Detection	Counts toward
`checkpoint`	Label begins with `CHECKPOINT`.	Target, not covered.
`observability`	Label begins with `Audit rollback:` or `Notifikasi rollback:`.	Neither target nor covered.
`llm_inferred`	`requires_manual_review === true` and not a checkpoint.	Target and covered.
`auto`	Everything else.	Target and covered.

```
targets  = auto + llm_inferred + checkpoint
covered  = auto + llm_inferred
coverage = targets == 0 ? 100 : round(covered / targets * 100)

checkpoint > 0     -> "PERLU INTERVENSI MANUAL"
llm_inferred > 0   -> "SIAP — VERIFIKASI KONFIGURASI"
otherwise          -> "SIAP DIEKSEKUSI"
```

Figure 7.3 — Coverage metric and status ladder. Observability actions are excluded from both numerator and denominator, since they are not reversals.

The summary is rendered two ways. A canvas comment is placed at (640, 90) containing the status, the coverage ratio, the four class counts, and a review instruction. The same headline is prepended to the workflow description so it is visible on the Shuffle workflow card without opening the editor:

```
[PERLU INTERVENSI MANUAL] Cakupan otomatis 3/5 (60%).
2 aksi tidak memiliki pembalik otomatis yang aman.
Dibuat otomatis dari '<source workflow name>'.
```

Each action flagged for manual review additionally receives its own comment, offset 320 px to its right. Checkpoint comments follow a four-part structure — identification, reason, impact if skipped, recommended remedy — with a severity derived from the application category:

Severity	Application pattern
`TINGGI` (high)	firewall, fortigate, palo, cisco, asa, edr, sophos, wazuh, crowdstrike, sentinel_one, network
`SEDANG` (medium)	entra, azure, okta, iam, identity, ldap, active_directory, google_workspace — and everything else, as the default

Severity granularity. Because the fallback is also `SEDANG`, the function has only two effective outcomes and an unrecognised application is rated identically to a known identity provider. As a display heuristic this is acceptable; it should not be presented as a risk model.

8. Validation Framework

`validateWorkflow(workflow, workflowId)` runs three levels in sequence and **short-circuits at the first level that produces errors**. A structurally invalid workflow is never checked semantically, which keeps the feedback to the LLM focused on one class of problem at a time.

8.1 Error Object Shape

```
{ level:    "structural" | "semantic" | "import",
  code:     "<CODE constant>",
  location: "actions[2].app_id",
  message:  "human-readable description",
  expected: "what a correct value looks like",   // optional
  received: "what was actually present" }        // optional
```

The `expected` field carries a concrete example rather than a type name — for instance, `"Unique UUID v4 for this action, e.g. '4a183af7-7cf2-4ce9-a008-a931c7cf4ff6'"`. This matters because these objects are fed back to the model as correction context; a worked example is more actionable than a schema fragment.

8.2 Level A — Structural

Pure JSON-shape validation with no external dependency.

Code	Checks
`INVALID_JSON`	Root is a non-array object.
`MISSING_FIELD`	`name` non-empty; `description` present (empty string allowed); `actions` a non-empty array; per action, all of `app_name`, `app_version`, `app_id`, `id`, `label`, `name` non-empty strings; per branch, `id`, `source_id`, `destination_id` present.
`INVALID_FORMAT`	Every action `id` and every branch `id` / `source_id` / `destination_id` matches the UUID pattern.
`INVALID_TYPE`	`is_start_node` boolean; `execution_delay` a number ≥ 0; `position` an object with numeric `x` and `y`; `branches` an array.
`DUPLICATE_ID`	Action ids unique; branch ids unique.
`MISSING_START_NODE` / `MULTIPLE_START_NODES`	Exactly one action has `is_start_node: true`.
`INVALID_BRANCH_REFERENCE`	Every branch endpoint references an existing action id.

Two early returns keep the error list interpretable: if the root is not an object, only that error is returned; if `actions` is missing or empty, action-level checks are skipped.

8.3 Level B — Semantic (Neo4j-backed)

Level B verifies that the workflow refers only to applications and actions that actually exist in the synchronised catalog. For each action with a non-empty `app_id`:

34. `MATCH (a:APP {app_id: $app_id})` — no result yields `INVALID_APP_ID`, and the action is skipped.
35. `app_name`, `app_version`, and `large_image` are compared by strict equality against the APP node. Each mismatch yields `APP_FIELD_MISMATCH` with the graph value as `expected`. The `large_image` error suppresses the received value in the message, since base64 image data would flood the error payload.
36. `MATCH (APP)-[:HAS_ACTION]->(act:ACTION_TEMPLATE {name: $action_name})` — no result yields `INVALID_ACTION_NAME`. The validator then issues a second query for up to five valid action names for that app and embeds them in `expected`, so the correction context contains real alternatives rather than a bare rejection.

> **Why this level exists.** It is the anti-hallucination barrier. A model that invents an application id, an action name, or a version cannot get past Level B, because every one of those values must match a row that was synchronised from Shuffle's own catalog. This is also why `getAppImages` injects `large_image` before validation — without it, every action would fail its own image comparison.

8.4 Level C — Reverse Mapping

Level C is the only check specific to the reverse-workflow domain. It queries the knowledge graph for every source action whose `HAS_REVERSE` status is `auto_mapped` with a non-empty inverse name, then asserts that for each such pair, the output contains either the expected inverse action name **or** the source action name.

```
MATCH (a:ACTION {workflow_id: $workflowId})-[:HAS_REVERSE]->(r:REVERSE_ACTION)
WHERE r.status = 'auto_mapped' AND r.reverse_action_name <> ''
RETURN DISTINCT r.reverse_action_name AS expected, a.action_name AS source
```

Accepting the source name is deliberate: an audit or notification action may legitimately be preserved rather than inverted (Section 7.2.1), and rejecting it would make Level C fight the assembler.

> **Weakened assertion.** Because the source name satisfies the check, a hypothetical output that merely copied every source action verbatim would pass Level C. The check therefore verifies presence, not inversion. With the current deterministic assembler this is harmless — the assembler never produces such an output — but the check should not be characterised as verifying reversal correctness.

Level C is skipped entirely when `workflowId` is null, which is the case for `POST /api/validate-workflow` calls that omit it.

8.5 Correction Instruction Synthesis

`buildCorrectionInstructions(errors)` converts the error array into a directive paragraph for the next LLM attempt. It opens with a count and the affected levels, then appends one targeted instruction per distinct error code present — for example, on `INVALID_APP_ID` or `APP_FIELD_MISMATCH` it instructs that app fields must be taken from the knowledge graph and not invented.

Combined with the structured `errors` array, this gives the model both the machine-readable failure list and a natural-language remediation directive, which is the mechanism by which the retry loop converges rather than repeating the same mistake.

8.6 Validation Helper Queries

Function	Purpose	Called from
`getAppImages(appIds)`	Fetches `large_image` per app id, so images can be injected before Level B compares them.	`generateWithRetry`, before validation.
`getReviewRequiredFromGraph(id)`	Returns all source actions with status `requires_manual_review` or `needs_llm`, from the graph.	`validateWorkflow` on success.
`getAutoMappedFromGraph(id)`	Returns full detail for `auto_mapped` pairs including parameters and images.	**Nowhere.** Exported and imported into `llmService` but never invoked.

All three swallow their own errors and return an empty result, so a transient Neo4j failure degrades enrichment rather than failing the request.

9. Data Model

9.1 Neo4j Node Labels

Label	Key property	Properties	Written by
WORKFLOW	workflow_id	workflow_name, description, start_node	saveGraph.js
ACTION	action_id	workflow_id, label, app_name, action_name, app_id, app_version, role, position (JSON), is_start, parameters (JSON)	saveGraph.js
REVERSE_ACTION	rev_id	source_action_id, source_action_name, reverse_action_name, app_name, app_id, status, reason	saveGraph.js
APP	app_id	app_name, app_version, large_image, sync_batch	saveAppsToNeo4j.js
ACTION_TEMPLATE	action_key	name, label, description, parameters (JSON), sync_batch	saveAppsToNeo4j.js
PARAMETER_TEMPLATE	parameter_key	parameter_name, required, parameter_type, description, sync_batch	saveAppsToNeo4j.js
APP_SYNC_META	id ("singleton")	last_synced_at, total_apps, last_batch_id	saveAppsToNeo4j.js

Composite keys are string-concatenated rather than structural: `action_key = "{app_id}_{action.name}"` and `parameter_key = "{action_key}_{param.name}"`. This is simple and works, but an app id or action name containing an underscore makes the key ambiguous under decomposition. Nothing in the code decomposes these keys, so the ambiguity is currently latent.

9.2 Neo4j Relationships

Type	From → To	Properties	Meaning
CONTAINS	WORKFLOW → ACTION	label	Workflow membership.
NEXT	ACTION → ACTION	condition, label	Source workflow control flow.
USES_APP	ACTION → APP	app_name	Which application an action invokes.
HAS_REVERSE	ACTION → REVERSE_ACTION	status	The persisted reverse resolution decision.
HAS_ACTION	APP → ACTION_TEMPLATE	—	Catalog: which actions an app exposes.
REQUIRES_PARAMETER	ACTION_TEMPLATE → PARAMETER_TEMPLATE	—	Catalog: action parameter schema.

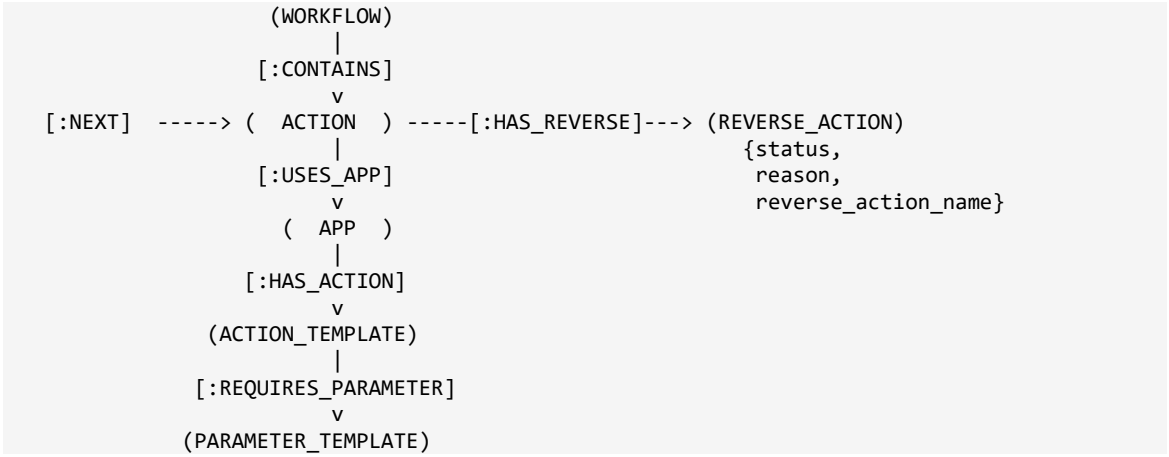


Figure 9.1 — Knowledge graph schema. The upper half is per-workflow; the lower half is the shared app catalog.

9.3 PostgreSQL Mirror

Tables are created idempotently with `CREATE TABLE IF NOT EXISTS` on every sync, and the whole write runs inside a single transaction with `ROLLBACK` on error.

Table	Primary key	Columns
`app_catalog`	`app_id`	`app_name`, `app_version`, `large_image`, `sync_batch`
`action_templates`	`action_key`	`app_id`, `action_name`, `action_label`, `description`, `parameters` (JSONB), `sync_batch`
`parameter_templates`	`parameter_key`	`action_key`, `parameter_name`, `required`, `parameter_type`, `description`, `sync_batch`

All writes are `INSERT ... ON CONFLICT DO UPDATE`, so the mirror converges to the latest catalog. `sync_batch` is a millisecond timestamp identifying the sync run. Rows from applications removed upstream are never deleted — the tables accumulate, and `sync_batch` is the only way to distinguish current from stale rows.

10. Logging and Observability

`utils/paperLog.js` produces structured console output at four pipeline points. The section prefixes map to the thesis results chapter, so a demonstration run yields transcribable evidence directly.

Function	Prefix	Emitted	Content
`logParsing`	`[4.2.3]`	After Stage 1	Markdown table: action id, app, action, summarised parameters, next actions.
`logPromptAndOutput`	`[4.2.5]`	After assembly	Truncated prompt, then a table of emitted actions with a manual-review flag column.
`logValidationStart`	`[4.2.6]`	After validation	PASS/FAIL per level with error codes, messages, and locations.
`logImportResult`	`[Import]`	After import	SUCCESS with workflow id and name, or FAILED with the reason.

Two defensive helpers keep output legible: `summarizeParams` skips any parameter named like `large_image`, truncates values at 36 characters and the whole summary at 80; `truncateMiddle` keeps the first 1000 and last 380 characters of a prompt and reports how many were omitted.

> **Gaps.** Logging is console-only, unstructured for machine consumption (no JSON lines, no correlation id), and has no severity levels beyond `console.log` / `warn` / `error`. There is no metrics endpoint and no request tracing. For a prototype whose logs are read by a human during a demonstration this is adequate; for operational use it is not.

A further caution: parameter values are printed to stdout. If a source workflow carries credentials or tokens in its parameters, they will appear in container logs. Only `large_image` is filtered.

11. Known Limitations and Design Trade-offs

This chapter is deliberate. Every item below was found by reading the shipped code, and each is stated with its actual consequence rather than minimised. Items are ordered by how likely they are to matter.

11.1 The workflow compiler is unused

`builders/buildShuffleWorkflow.js` exports three functions. Only `importWorkflowToShuffle` is called. `buildShuffleWorkflow(plan)` — 232 lines that expand a plan into the complete Shuffle workflow object with all ~50 platform fields — and `buildLinearConnections(steps)` are never invoked anywhere in the service.

The two are also shape-incompatible with the pipeline: the compiler expects `plan.steps[]` with `step_id`, `action_name`, and `purpose`, whereas `normalizeWorkflowIds` produces `workflow.actions[]` with `id`, `name`, and `label`. What is actually POSTed to Shuffle is the assembled object — `{ name, description, start, actions, branches, comments }` — and Shuffle populates the remaining fields itself on import.

> **Recommendation.** Either delete the compiler or wire it in. As it stands, a reader tracing the code would reasonably assume it is on the critical path, and a reviewer who notices it is unused will ask why. If it is retained as scaffolding for a planned refactor, say so in a header comment.

11.2 LLM candidates are matched by application, not by action

Described in Section 7.5. `pool.findIndex(a => a.app_name === node.app_name)` followed by `splice` means candidate-to-source binding is positional within an application group. With two `needs_llm` actions from the same app, the reversal generated for one can be attached to the other.

Mitigations already present: every such action is flagged `requires_manual_review`, receives a canvas annotation instructing verification, and appears in the API `review_required` list. A mis-bound reversal is therefore visible to the analyst rather than silently executed.

Fix: have the llm-service return `source_action_id` on each candidate and match on that, falling back to app matching only when the id is absent.

11.3 Graph write errors are swallowed

`saveGraphToNeo4j` catches, logs, and returns normally. The failure surfaces two stages later as `Workflow context not found in Neo4j`, which points at the reader rather than the writer. Rethrowing would cost nothing and would make the 500 response name the real fault.

11.4 Catalog sync runs after graph persistence

`USES_APP` relationships are written before the APP nodes may exist. When they do not, the Cypher `MATCH` fails silently and the relationship is simply absent — with no error anywhere. The boot-time sync usually masks this. Moving `safeSyncApps()` above `saveGraphToNeo4j()` in the handler resolves it.

11.5 The LLM token is cached and never refreshed

`_token` is set once and reused for the process lifetime. There is no expiry handling and no re-login on 401. Once the token expires, every generation fails until the container restarts. The retry loop will consume all three attempts against the same expired token. A 401-triggered re-login would be a small change.

11.6 Toggle reversal has broad reach

The toggle test precedes knowledge-base resolution and fires on any parameter named `enabled`, `active`, `blocked`, `suspended`, and similar. An action classified `requires_manual_review` that happens to carry such a parameter is auto-reversed and **not** flagged for review, because `buildToggleReverse` does not set `requires_manual_review`.

This is correct and valuable for the enable/disable pattern it was written for. It is worth deciding explicitly whether the toggle should defer to an explicit `requires_manual_review` listing, or whether toggled actions should carry the review flag.

11.7 Checkpoints depend on Shuffle Tools being present

Described in Section 7.6. If the source workflow contains no Shuffle Tools action, `buildCheckpoint` returns `null` and irreversible actions produce no canvas node. The `review_required` list still records them, so the API contract is intact, but the visual warning — which is the primary safety affordance of the design — is absent exactly when it is needed.

Fix: resolve the Shuffle Tools `app_id` and `app_version` from the APP catalog in Neo4j rather than from the source workflow. The catalog is already synchronised and queried elsewhere in the same module.

11.8 No authentication on the service's own endpoints

All four endpoints are unauthenticated and `cors()` is applied with no origin restriction. Anyone who can reach port 5005 can submit workflows and cause writes to Neo4j and imports into Shuffle. The service also holds Shuffle and llm-service credentials. Deployment on an isolated Compose network is the current control; that should be stated as a deployment requirement rather than assumed.

Related: `LLM_AUTH_USER` and `LLM_AUTH_PASS` default to `admin`/`admin`. Defaults that are valid credentials are worse than defaults that fail loudly.

11.9 Smaller items

Item	Detail	Impact
Reverse workflows are always linear	`normalizeWorkflowIds` discards branch structure and rebuilds a sequential chain.	Design limit. Conditional rollback is not expressible.
Synchronous long request	The endpoint blocks for the full pipeline, with an LLM timeout of up to 16 minutes.	Any intermediary with a shorter timeout will drop the connection. The status endpoint exists but cannot be reached usefully because the pipeline is not backgrounded.
In-memory status store	A `Map` with no eviction, lost on restart, not shared across replicas.	Unbounded growth; incorrect results behind a load balancer.

Item	Detail	Impact
Hard-coded port	`app.listen(5005)`; no `PORT` variable.	Requires a code edit or port mapping.
All errors return 500	No 400 path on the generation endpoint.	Client faults are indistinguishable from server faults.
`role` property is dead	Written by `inferRole`, read by nothing.	Misleading to a reader; harmless at runtime.
`reversible_prefix_pairs` is inert	Defined in the knowledge base, read by no code in this service.	Should not be described as an active mechanism.
`getAutoMappedFromGraph` unused	Imported into `llmService` but never called.	Dead import.
`lookupReverse` / `isIrreversible` unused	Exported from `reverseMap.js`, called nowhere.	Dead exports.
nodemon is a runtime dependency	Listed under `dependencies`; `npm install --omit=dev` therefore still installs it.	Larger image; a development tool ships to production.
`uuid` and `crypto.randomUUID` coexist	`builders/` uses the package, `llm/` uses the built-in.	Redundant dependency.
Parameters logged to stdout	Only `large_image` is filtered from `summarizeParams`.	Credentials in workflow parameters would appear in container logs.
No automated tests	No test framework, test files, or test script in `package.json`.	Regressions in the assembly logic would not be caught mechanically.
No graceful shutdown	No SIGTERM handler; driver and pool are never closed.	Connections are dropped rather than drained on container stop.

11.10 What the prototype does not claim

- It does not claim that generated reverse workflows are safe to execute unreviewed. Every output is labelled a draft and carries an explicit coverage status.
- It does not claim complete application coverage. Eighteen applications are curated; anything outside them falls to heuristics and then to `needs_llm`.
- It does not claim that the LLM authors the workflow. Model output is a bounded candidate pool consumed only where rule-based resolution explicitly deferred, and always flagged for verification.
- It does not claim to reconstruct pre-execution state. Actions whose reversal requires state that was never captured are routed to manual review by design, not attempted.

12. Operations and Troubleshooting

12.1 Bringing the Service Up

37. Confirm `.env` exists **one directory above** the service root and carries the Neo4j, Shuffle, llm-service, and PostgreSQL settings from Table 3.1.
38. Start Neo4j, PostgreSQL, Shuffle, and llm-service first. The service starts regardless, but the first request will fail without them.
39. Start the service and watch stdout for `[neo4j] Connected`, `[apps] TOTAL = n`, and `Reverse Workflow Service running on port 5005`. A cold start against an empty Neo4j will pause on the catalog sync before the port is bound.
40. Verify liveness with `GET /`, then confirm the catalog with `MATCH (a:APP) RETURN count(a)` in Neo4j Browser.
41. Submit a source workflow to `POST /api/reverse-workflow` and read the `[4.2.3]` … `[Import]` log blocks in order.

12.2 Troubleshooting

Symptom	Likely cause	Action
`Workflow context not found in Neo4j`	The graph write failed and was swallowed (Section 11.3), or Neo4j is unreachable.	Look for `Neo4j Save Error` earlier in the log, which carries the real cause. Verify `NEO4J_URI` and credentials.
`[LLM] Login failed`	llm-service down, or `LLM_AUTH_USER` / `LLM_AUTH_PASS` wrong.	Confirm the service responds at `{LLM_API_URL}{LLM_API_PREFIX}/auth/login`.
Generation fails only after long delay	The 16-minute default timeout elapsed.	Lower `LLM_CALL_TIMEOUT_MS` for faster feedback; check llm-service load and model residency.
Every attempt fails with 401 from llm-service	The cached token expired (Section 11.5).	Restart the container. There is no in-process recovery.
Validation fails with `INVALID_APP_ID`	The app catalog is stale or empty.	Check `[apps]` log lines; force a resync by deleting the `APP_SYNC_META` node or lowering `APP_SYNC_TTL_HOURS`.
Validation fails with `APP_FIELD_MISMATCH` on `large_image`	`getAppImages` failed, so images were not injected.	Look for `[LLM] inject large_image gagal`. Confirm APP nodes carry `large_image`.
`INVALID_ACTION_NAME`	The LLM proposed an action the app does not expose.	The `expected` field lists up to five valid names. Persistent recurrence indicates a prompt or retrieval problem in llm-service.
`no_auto_reverse` on a workflow that clearly changed state	Every action classified `read` or `utility`.	Check the `method` parameter — a `GET` value forces the `read` classification. Verify the app is present in `reverseActionMap.json`.
Irreversible actions produce no canvas node	The source workflow had no Shuffle Tools action (Section 11.7).	Consult the `review_required` field in the API response, which still records them.

Symptom	Likely cause	Action
`[importer] SHUFFLE_API_KEY tidak di-set`	Missing or empty environment variable.	Set it; the importer throws before any HTTP call.
Shuffle rejects the import	Field or structural rejection at the platform level.	The rejection body is logged (first 300 characters) and re-fed to the LLM as `SHUFFLE_IMPORT_ERROR`.
`[APP] sync skipped`	Shuffle unreachable or credentials rejected.	Non-fatal. The pipeline continues on the cached catalog; it will fail at Level B if the cache is empty.

Table 12.1 — Troubleshooting reference.

12.3 Extending the Knowledge Base

Adding an application requires no code change — only a new entry in `config/reverseActionMap.json` and a restart, since the file is read once at module load.

```
"New_Application": {
  "reversible": [
    { "action": "post_create_thing", "reverse_action": "delete_thing" }
  ],
  "requires_manual_review": [ "put_update_thing" ],
  "no_reverse_needed":      [ "post_get_thing" ],
  "needs_llm":              [ "patch_toggle_thing" ],
  "_note": "why these decisions were made"
}
```

- Action names must match the values Shuffle reports, before normalisation — matching lowercases and converts whitespace and hyphens to underscores, but does nothing else.
- List read actions explicitly under `no_reverse_needed` whenever the connector prefixes them unconventionally, as Microsoft Graph does with `post_..._get_user`.
- Prefer `requires_manual_review` over an approximate inverse. A checkpoint costs an analyst thirty seconds; a wrong reversal costs an outage.
- Record the reasoning in `_note`. The existing entries do this consistently and it is what makes the curation reviewable.

Appendix A — Worked Example

A four-action source workflow illustrating each resolution status.

#	App	Action	Classification	Resolution	Outcome
1	Shuffle Tools	`repeat_back_to_me`	`utility`	—	Skipped.
2	Microsoft_Graph	`post_..._get_user`	`read`	—	Skipped.
3	FortiGate_Firewall	`post_add_firewall_address`	`containment`	`auto_mapped`	Emit `delete_firewall_address` with source parameters.
4	Slack	`post_chat_postmessage`	`notification`	—	Preserved with rollback content, appended to tail.

Assembly iterates in reverse (4, 3, 2, 1). Action 4 is deferred to the observability tail; action 3 produces the containment reversal; actions 2 and 1 are skipped. The final list is `[delete_firewall_address, post_chat_postmessage]` — the reversal first, the notification last.

```
counts    = { auto: 1, llm_inferred: 0, checkpoint: 0, observability: 1 }
targets   = 1 + 0 + 0 = 1
covered   = 1 + 0     = 1
coverage  = 100%
status    = "SIAP DIEKSEKUSI"

description:
  [SIAP DIEKSEKUSI] Cakupan otomatis 1/1 (100%). Seluruh aksi
  containment dibalik otomatis. Dibuat otomatis dari '<name>'.
```

Now substitute action 3 with `Palo_Alto_Networks / post_block_outbound_ip`, which the knowledge base lists under `requires_manual_review` because the connector exposes no unblock operation. The containment slot becomes a checkpoint instead:

```
counts    = { auto: 0, llm_inferred: 0, checkpoint: 1, observability: 1 }
targets   = 1,  covered = 0,  coverage = 0%
status    = "PERLU INTERVENSI MANUAL"

canvas comment on the checkpoint node:
  CHECKPOINT: 'post_block_outbound_ip' (Palo_Alto_Networks)
  ALASAN: tidak ada pembalik otomatis yang aman — pembalikan butuh
          kondisi sebelum eksekusi (pre-state) yang tidak tersimpan.
  DAMPAK bila dilewati: efek aksi sumber TETAP AKTIF. Risiko: TINGGI.
  REKOMENDASI: pulihkan konfigurasi Palo_Alto_Networks secara manual
          ke kondisi sebelum eksekusi, lalu verifikasi hasilnya.
```

The severity is `TINGGI` because the application name matches the firewall pattern in `deriveSeverity`. The `review_required` array in the API response carries the same entry, so the caller can act on it without parsing the canvas.

Appendix B — Function Index

Function	Module	Called from
parseWorkflow	parsers/workflowParser.js	server.js
buildGraph	graph/graphBuilder.js	server.js
inferRole	graph/graphBuilder.js	buildGraph (result unused downstream)
saveGraphToNeo4j	neo4j/saveGraph.js	server.js
getWorkflowContext	neo4j/queryWorkflowContext.js	server.js (existence check only)
syncAppCatalog	shuffleApps/syncAppCatalog.js	server.js via safeSyncApps
checkSyncNeeded	shuffleApps/syncAppCatalog.js	syncAppCatalog
fetchAllApps	shuffleApps/fetchApps.js	syncAppCatalog
saveAppsToNeo4j	shuffleApps/saveAppsToNeo4j.js	syncAppCatalog
saveAppsToPostgres	shuffleApps/saveAppsToPostgres.js	syncAppCatalog
resolveReverse	config/reverseMap.js	buildGraph, assembleReverse
lookupReverse	config/reverseMap.js	**unused**
isIrreversible	config/reverseMap.js	**unused**
login / getToken	llm/llmService.js	callLLM
callLLM	llm/llmService.js	generateWithRetry
parseWorkflowJSON	llm/llmService.js	generateWithRetry
classifyAction	llm/llmService.js	assembleReverse
isReadOnlySource	llm/llmService.js	assembleReverse
buildToggleReverse	llm/llmService.js	assembleReverse
buildCheckpoint	llm/llmService.js	assembleReverse
withRollbackContent	llm/llmService.js	buildObservabilityAction
buildObservabilityAction	llm/llmService.js	assembleReverse
assembleReverse	llm/llmService.js	generateWithRetry
normalizeWorkflowIds	llm/llmService.js	generateWithRetry
classifyEmitted	llm/llmService.js	buildRollbackSummary, buildComments
buildRollbackSummary	llm/llmService.js	buildComments
deriveSeverity	llm/llmService.js	buildComments
buildComments	llm/llmService.js	generateWithRetry

Function	Module	Called from
generateWithRetry	llm/llmService.js	server.js
validateStructure	validators/validateWorkflow.js	validateWorkflow
validateSemantic	validators/validateWorkflow.js	validateWorkflow
validateReverseMapping	validators/validateWorkflow.js	validateWorkflow
buildCorrectionInstructions	validators/validateWorkflow.js	validateWorkflow
validateWorkflow	validators/validateWorkflow.js	server.js, generateWithRetry
buildImportError	validators/validateWorkflow.js	generateWithRetry
getAppImages	validators/validateWorkflow.js	generateWithRetry
getReviewRequiredFromGraph	validators/validateWorkflow.js	validateWorkflow
getAutoMappedFromGraph	validators/validateWorkflow.js	**unused**
importWorkflowToShuffle	builders/buildShuffleWorkflow.js	generateWithRetry
buildShuffleWorkflow	builders/buildShuffleWorkflow.js	**unused**
buildLinearConnections	builders/buildShuffleWorkflow.js	**unused**
logParsing … logImportResult	utils/paperLog.js	server.js, generateWithRetry

End of document.